

- ✓ Logs outputs and errors in `./logs/`
- ✓ Displays colored pass/fail results
- ✓ Cleans up afterward

Would you like me to extend this script to also **automatically generate a few sample** `.cub` **maps** for testing (valid + invalid)?

That way you can test parser robustness too.



give me separate things i need to test

Excellent question 🔥 — for a **cub3D** project (raycasting + parsing + rendering), you'll want to test **each subsystem separately**, because the project has multiple potential failure points.

Below is a **complete test checklist** — organized into **mandatory functional categories**.

🧭 1. Map Parsing Tests

These verify that your `.cub` file parser works as expected.

✓ Valid cases

Test	Description
<code>valid_basic.cub</code>	Simple closed map with valid textures and colors.
<code>spaces_in_map.cub</code>	Map that includes spaces inside the map lines (should be handled correctly).
<code>mixed_order.cub</code>	Elements (NO, SO, WE, EA, F, C) appear in random order.
<code>extra_newlines.cub</code>	Extra empty lines between sections.
<code>valid_multiple_spaces.cub</code>	Multiple spaces between identifiers and paths.

✗ Invalid cases (should print Error\n + message)

Test	Description
missing_texture.cub	One of NO/SO/WE/EA is missing.
invalid_texture_path.cub	Texture path doesn't exist.
invalid_rgb_values.cub	RGB values not in [0, 255] .
bad_identififier.cub	Wrong identifiers like "NORTH" instead of "NO".
map_not_closed.cub	Open map (holes or borders not surrounded by walls).
multiple_players.cub	More than one player (multiple N/S/E/W).
no_player.cub	No player start position.
non_rectangular_map.cub	Rows of different lengths.
invalid_characters.cub	Unexpected character (like x or 2).
empty_file.cub	File is empty.
wrong_extension.txt	Wrong file extension (not .cub).



2. Color and Texture Parsing

Test	Description
valid_colors.cub	Proper RGB for F and C.
leading_zeros_rgb.cub	Values like F 001, 255, 000 (should still work).
extra_spaces_colors.cub	Extra spaces like F 220 , 100 , 0 .
missing_rgb_values.cub	Missing one or more RGB components.
non_numeric_rgb.cub	Contains letters (e.g., F 200, a, 50).

3. Raycasting & Rendering

You can't "unit test" visual output automatically, but you can **observe** expected behavior.

Test	Description
Simple closed map	3D projection should render walls properly.
Rotation	Arrow keys rotate view correctly (smoothly).
Movement	WASD movement corresponds to directions.
Collision (bonus)	Player cannot move through walls.
Different textures	Each wall direction (N/S/E/W) uses correct texture.
Floor & ceiling colors	Properly displayed, not flickering.

4. Event Handling

Event	Expected behavior
ESC key	Cleanly exit program.
Click red cross (window close)	Cleanly exit without leaks.
Minimize and restore window	No crash, image restores.
Arrow keys	Rotate left/right correctly.
W/A/S/D keys	Move forward/back/strafe smoothly.

5. Error Handling and Stability

Case	Expected behavior
Missing file argument	Print "Error\n" and message, exit cleanly.

Invalid file path	Proper error message.
Double free or segfault	Should never happen.
Memory leaks	None (check with <code>valgrind</code>).
Window handling	Smooth open/close, no memory leaks on exit.



6. Bonus Tests (if implemented)

Feature	Tests
Wall collisions	Can't walk through walls.
Minimap	Correctly displays layout and player position.
Doors	Opens/closes properly.
Mouse rotation	Works without lag or stutter.
Animated sprites	Smoothly animated.
HUD or weapon	Renders and updates correctly.



Optional: Automated Script Grouping

You can place all test `.cub` files in folders:

```
pgsql
```

[Copy code](#)

```
tests/  
├─ valid/  
├─ invalid/  
├─ colors/  
└─ bonus/
```

Then extend your `bash.sh` to automatically run all `.cub` files in these directories and

verify which ones succeed/fail.

Would you like me to generate a **set of example** `.cub` **files** for these test categories (both valid and invalid)?

That way you could plug them directly into your test folder and run the `bash.sh` script.

